



Many hands make light work: An LLM-based multi-agent system for detecting malicious PyPI packages

Muhammad Umar Zeshan , Motunrayo Ibiyo , Claudio Di Sipio , Phuong T. Nguyen ,
Davide Di Ruscio *

Universit'a degli studi dell'Aquila, 67100, L'Aquila, Italy

ARTICLE INFO

Editor: Dr T. Mårtensson

Keywords:

Malicious PyPI package

LLMs

Multi-agent systems

ABSTRACT

Malicious code in open-source repositories such as PyPI poses a growing threat to software supply chains. Traditional rule-based tools often overlook the semantic patterns in source code that are crucial for identifying adversarial components. Large language models (LLMs) show promise for software analysis, yet their use in interpretable and modular security pipelines remains limited.

This paper presents LAMPS, a multi-agent system that employs collaborative LLMs to detect malicious PyPI packages. The system consists of four role-specific agents for *package retrieval*, *file extraction*, *classification*, and *verdict aggregation*, coordinated through the CrewAI framework. A prototype combines a fine-tuned CodeBERT model for classification with LLaMA 3 agents for contextual reasoning. LAMPS has been evaluated on two complementary datasets: D_1 , a balanced collection of 6000 `setup.py` files, and D_2 , a realistic multi-file dataset with 1296 files and natural class imbalance. On D_1 , LAMPS achieves 97.7% accuracy, surpassing MPHunter and TD-IDF stacking models—two state-of-the-art approaches. On D_2 , it reaches 99.5% accuracy and 99.5% balanced accuracy, outperforming RAG-based approaches and fine-tuned single-agent baselines. McNemar's test confirmed these improvements as highly significant. The results demonstrate the feasibility of distributed LLM reasoning for malicious code detection and highlight the benefits of modular multi-agent designs in software supply chain security.

1. Introduction

Open-source platforms foster software development by enabling programmers to share and access extensive collections of reusable code (Robillard et al., 2014). This practice helps developers accelerate their work and increase productivity. Unfortunately, *every rose has its thorn*: the availability of a considerable amount of reusable components also creates opportunities for adversarial users to inject malicious code into widely used libraries, posing significant security risks to software supply chains (Huang et al., 2024; Yang et al., 2022). Malicious packages are deliberately disguised as legitimate code to trick users, making detection particularly challenging (Alfadel et al., 2023). As a result, recognizing malicious code is becoming increasingly critical, especially with the proliferation of AI-based systems that are trained on publicly available repositories.

A variety of approaches have been developed to automatically analyze security threats, including static rule-based scanners

(Ruohonen et al., 2021a), signature-driven classifiers (Chakraborty et al., 2020), clustering methods (Liang et al., 2023), and behavior sequence models (Zhang et al., 2025). While these methods achieve encouraging performance, they often fail in the presence of obfuscation, indirect API usage, or logic-level concealment. The recent success of large language models (LLMs) has opened new opportunities for analyzing and reasoning about software systems. Their ability to interpret code semantics through natural language prompts makes them attractive for tasks such as code summarization, defect detection, and vulnerability identification (Ahmad et al., 2021; Chen et al., 2021). Despite this potential, applications in security-critical domains such as malicious code detection in open-source repositories remain limited. Moreover, many existing attempts treat LLMs as monolithic black boxes, where a single model instance is tasked with multiple stages of analysis without meaningful modularity or transparency (Ohm et al., 2020; Wang et al., 2023). Empirical evidence on Retrieval Augmented Generation (RAG) further indicates that retrieval alone does not reliably improve the detection of

* Corresponding author.

E-mail addresses: muhammadumar.zeshan@student.univaq.it (M.U. Zeshan), motunryoosatohanmen.ibiyo@student.univaq.it (M. Ibiyo), claudio.disipio@student.univaq.it (C. Di Sipio), phuong.nguyen@univaq.it (P.T. Nguyen), davide.diruscio@univaq.it (D. Di Ruscio).

<https://doi.org/10.1016/j.jss.2026.112792>

Received 20 September 2025; Received in revised form 19 December 2025; Accepted 17 January 2026

Available online 21 January 2026

0164-1212/© 2026 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

malicious intent in PyPI packages (Ibiyo et al., 2025). Altogether, this motivates the development of an approach that can reliably surface file-level signals and provide auditable package-level decisions.

Recently, the software engineering community has witnessed the emergence of multi-agent systems that leverage orchestrations of LLMs to decompose complex tasks into specialized agents, often surpassing single-agent pipelines (He et al., 2025; Nguyen et al., 2025). While these systems show promise for coding tasks, there remains a need to investigate coordination patterns, the use of shared context, and the feasibility of LLM-based multi-agent designs for security analysis.

In this work, we propose LAMPS, a novel LLM-based multi Agent system for detecting Malicious PyPI PackageS. We decompose the detection task into a coordinated network of specialized agents with clearly defined roles and prompts, orchestrated through CrewAI (Taulli et al., 2025). Instead of a monolithic pipeline, responsibilities such as package retrieval, file filtering, code-level classification, and verdict aggregation are distributed across autonomous units that communicate through structured prompt-based messages. Classifier Agent uses a fine-tuned CodeBERT model trained on labeled Python files, while the other agents rely on LLaMA-3 for role-specific reasoning. This design reflects a shift from singular model invocation to structured, distributed reasoning, and it is supported by a clarified threat and detection scope that focuses on learned code-level indicators.

Although this work explores a new direction, it is based on our previous work on the detection of malicious packages (Ibiyo et al., 2025). We further extend the original framework using an LLM-based multi-agent system, and conducting an empirical evaluation with real-world datasets. We describe agent roles and communications, construct and validate datasets for two settings, adopt a rigorous evaluation protocol with package-level splits and repeated runs, test statistical significance with McNemar’s test, and report standard metrics alongside balanced accuracy in imbalanced settings. We also record efficiency measurements on our local environment to complement accuracy-based comparisons. LAMPS has been evaluated against recent baselines, including MPHunter (Liang et al., 2023) and RAG-based configurations (Ibiyo et al., 2025), using the same datasets and experimental settings.

The key contributions of this paper are as follows:

- **Solution.** We introduced LAMPS, a multi-agent LLM-based system for detecting malicious code in open-source Python repositories. The design leverages role specialization for package retrieval, file filtering, and file-level classification with a fine-tuned CodeBERT, as well as conservative package-level aggregation.
- **Evaluation.** Using two datasets with 6000 and 1,296 samples, we evaluated LAMPS against state-of-the-art baselines, including MPHunter (Liang et al., 2023), a TF-IDF-based stacking model (Samaana et al., 2025) working on top of machine learning and static analysis, and RAG-based approaches (Ibiyo et al., 2025), following an explicit protocol with package-level splits, repeated runs with fixed seeds, and significance testing. Through the evaluation, we also demonstrate that the application of LLM-based multi-agent systems is advantageous compared to that of single LLMs.
- **Open Science.** We released a replication package with full source code, data preparation scripts, prompts, and environment specifications to facilitate independent verification (Zeshan et al., 2025).

Structure. The remainder of this paper is organized as follows. Section 2 presents the background and motivation for the study. Section 3 describes the design of LAMPS, including agent responsibilities, coordination, and threat model. Section 4 provides the proof of concept, which also includes dataset construction. Section 5 presents the experimental setup, results for three research questions, and a discussion of findings. Section 6 reviews related work. Finally, Section 7 concludes the paper and outlines directions for future research.

2. Motivation and background

This section introduces the problem context and highlights the motivations that guided the design of our approach. We first describe how malicious code typically manifests in Python package ecosystems and why it poses unique detection challenges. Afterwards, two motivating examples are introduced to illustrate how hidden patterns could hinder the detection of malicious PyPI packages. We then outline the role of LLMs in capturing semantic properties of code, and discuss how a multi-agent architecture provides practical and conceptual benefits over a monolithic pipeline. Together, these elements establish the foundation for the methodology detailed in the next section.

2.1. Motivating examples

The Python Package Index (PyPI) is a widely used repository that allows developers to distribute source archives containing both library code and installation scripts. A distinctive characteristic of this ecosystem is the reliance on files such as `setup.py`, which can execute arbitrary Python code during installation. This flexibility, while powerful for developers, creates an attack surface that adversaries can exploit by embedding harmful logic inside otherwise legitimate-looking packages.

Standard attacker techniques include typosquatting on popular package names to trick users into installing a malicious variant, inserting encoded strings to conceal commands, invoking `subprocess` calls to fetch and execute remote payloads, and performing file or network operations during installation or at first import. Because such behaviors are interleaved with benign functionality, they are difficult to detect through simple pattern matching or metadata checks.

A motivating example is shown in Listing 3, taken from the package `pongreplace-10.4`, where the `setup.py` script constructs a base64-encoded command and executes it via a hidden `subprocess` call. The obfuscation ensures that surface-level similarity measures cannot easily distinguish the script from benign installers. This illustrates why detection must move beyond syntactic features to capture semantic cues of malicious behavior.

Fig. 1 complements this example by visualizing the supply-chain path from an adversarial upload to downstream compromise and highlighting the points where malicious logic can hide in installation scripts or auxiliary modules. The figure motivates the need for semantics-aware, auditable analysis that preserves file-level signals and supports package-level decisions.

Another observation is that malicious logic is not always confined to installation scripts. In multi-file packages, payloads can be distributed across auxiliary modules, with only a small fraction of files exposing suspicious patterns. A detector that operates only at the file level risks underestimating package-level risk.

Listing 2 shows a malicious `setup.py` script extracted from the PyPI package named `esqpepywvirtual-9.5`. The script conditionally invokes a hidden PowerShell command¹ that downloads and executes a remote payload, thereby compromising the host system during installation. This payload is obfuscated using a `base64-encoded` command and executed silently to evade detection. The package is disguised as a typical utility library, making it difficult to detect using traditional rule-based tools.

MPHunter (Liang et al., 2023) is a state-of-the-art approach in detecting malicious PyPI packages, and we used it to test with the code in Listing 2. The result showed that MPHunter fails to detect the hidden malicious intent, i.e., it cannot recognize the silently `base64-encoded` command. This is consistent with a clustering-based mechanism that groups samples by surface-level similarity: obfuscated, single-file installer logic

¹ The PowerShell payload in Listing 2 has been truncated for the sake of presentation. The full scripts are available in the online appendix (Zeshan et al., 2025).

```
import subprocess, os
if not os.path.exists('tahg'):
    subprocess.Popen(
        'powershell -WindowStyle Hidden -EncodedCommand '
        'cABvAHcAZQByAHMAaABLAGwAbAAgAEKAbgB2AG8AawBlAC0AVwBlA...',
        shell=False,
        creationflags=subprocess.CREATE_NO_WINDOW)
```

Listing 1. Excerpt from the malicious `setup.py` of package `pongreplace-10.4`, showing the obfuscated payload execution.

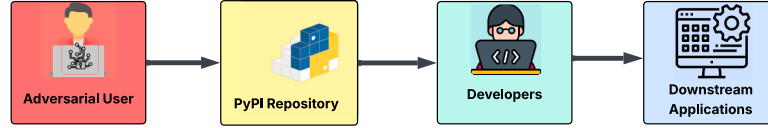


Fig. 1. Motivating example of supply-chain risk in PyPI. An adversarial upload can embed obfuscated logic in `setup.py` or auxiliary modules; developers unknowingly install the package, and downstream applications are affected at runtime. The example motivates semantics-aware analysis and conservative package-level aggregation.

```
from distutils.core import setup

try:
    import subprocess
    import os
    if not os.path.exists('tahg'):
        # obfuscated payload (base64 encoded command)
        subprocess.Popen(
            'powershell -WindowStyle Hidden -EncodedCommand <truncated>',
            shell=False,
            creationflags=subprocess.CREATE_NO_WINDOW
        )
except:
    pass

try:
    setup(
        name='pongreplace',
        packages=['modlib'],
        version='10.4',
        description='A library for creating a terminal user interface',
        author='EsqueleSquad',
        author_email='tahgofficial@proton.me',
        classifiers=[
            'Programming Language :: Python :: 3'
        ],
    )
except:
    pass
```

Listing 2. Obfuscated malicious code in `esquepyvirtual-9.5`.

may not exhibit the frequency-based characteristics required to form a clear outlier cluster.

These examples motivate us to come up with an approach that can recognize the hidden intent by means of a series of LLM-based agents. This leads to the conservative aggregation strategy in our proposal: if any file is flagged as malicious, the package is treated as malicious. Such a policy reduces false negatives at the ecosystem level, where the cost of a missed detection is typically higher than the cost of a false alarm.

These considerations shape the design of our pipeline: (i) provenance-preserving retrieval and deterministic extraction to ensure that the correct inputs are analyzed; (ii) file-level classification using a semantics-aware model capable of recognizing encoded payloads, process invocation, and unauthorized I/O; and (iii) package-level aggregation to translate heterogeneous file-level outputs into an auditable decision. These elements are elaborated in [Section 3](#) and empirically evaluated in [Section 4](#).

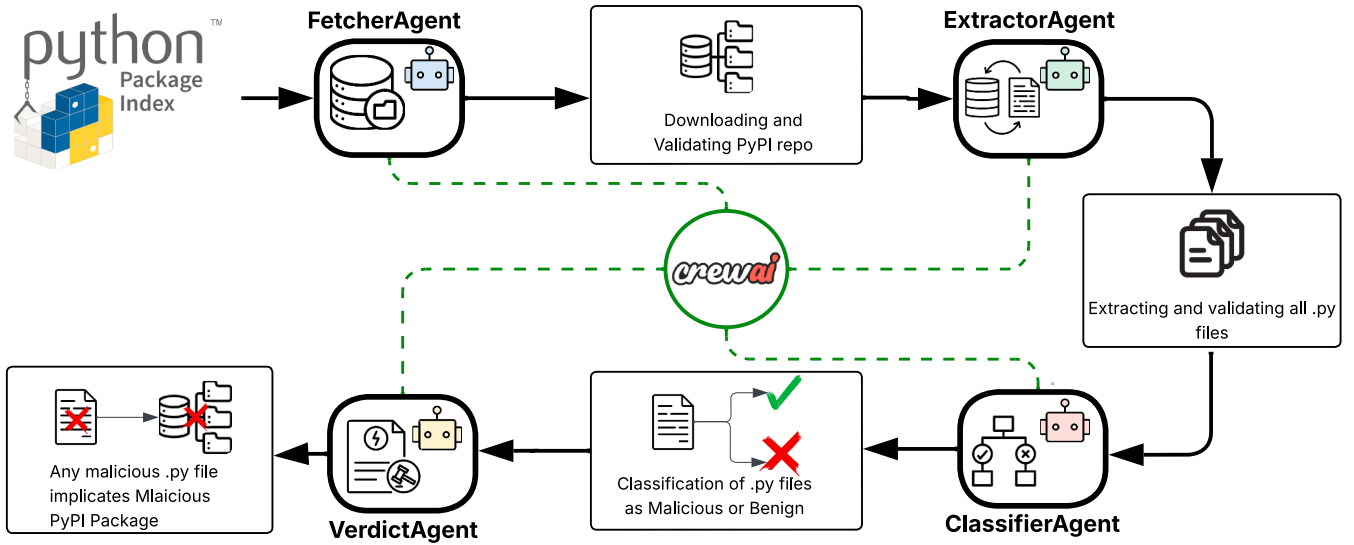


Fig. 2. Architecture of LAMPS showing agent responsibilities and communication flow.

2.2. Large language models for code understanding

LLMs have recently demonstrated strong capabilities in capturing the semantic properties of both natural and programming languages. When trained on large-scale code corpora, such models can embed source code into vector spaces that preserve syntactic structure and semantic intent, making them suitable for downstream tasks such as code summarization, defect detection, and vulnerability identification (Allamanis et al., 2018; Chen et al., 2021). Unlike traditional feature engineering or rule-based approaches, LLMs learn distributed representations that generalize across projects and programming idioms.

For our purposes, the following two observations are critical:

1. *General-purpose LLMs alone are not sufficient to distinguish malicious from benign code reliably.* Prior work shows that zero-shot or few-shot prompting without domain-specific adaptation often leads to unstable or superficial predictions in security-sensitive contexts. This limitation motivates the use of a model pre-trained on source code and then fine-tuned on a curated dataset of benign and malicious files. In LAMPS, we adopt CodeBERT as the classifier backbone because it has been explicitly pre-trained for program understanding and can be adapted through supervised training to capture behavioral signatures such as encoded payload decoding, process invocation, and unauthorized I/O.
2. *While file-level classifiers are necessary for detecting fine-grained malicious patterns, they must operate within broader workflows where multiple files and package metadata interact.* Models like LLaMA-3 offer complementary strengths, as they are capable of prompt-based reasoning and flexible input handling, making them well-suited for orchestrating auxiliary tasks such as package retrieval, file filtering, and verdict explanation. By combining a fine-tuned model specialized for classification (CodeBERT) with role-specific reasoning agents powered by a general-purpose LLM (LLaMA-3), our architecture balances accuracy at the classification step with flexibility in pipeline coordination.

This design choice addresses a central challenge in malicious package detection: the need to preserve semantic sensitivity at the code level while enabling modular and auditable reasoning across the entire package. The resulting system leverages LLMs not as a monolithic solution

but as specialized components integrated into a coordinated workflow, a strategy that we evaluate in detail in Section 5.

2.3. Multi-agent orchestration and motivation

Although LLMs offer the semantic sensitivity needed for code analysis, relying on a single model instance as an end-to-end detector introduces both practical and conceptual limitations. A monolithic prompt that ingests all package content must simultaneously retrieve archives, extract files, classify source code, and provide justifications, without guarantees that each step is executed reliably. In multi-file packages, this approach is further constrained by context window limits, token truncation, and order sensitivity: malicious logic can be overlooked if a critical file is omitted or its signals diluted among benign content. For example, a package may contain dozens of benign utility modules alongside a `setup.py` script that silently downloads and executes a payload. A monolithic model truncated to fit within the context window may exclude the `setup.py` file entirely, misclassifying the package as benign. In contrast, a multi-agent system ensures that such critical files are always extracted, classified, and considered in the final verdict.

A multi-agent architecture mitigates these challenges by decomposing the overall workflow into specialized roles. Separate agents for fetching, extraction, classification, and verdict aggregation operate with clearly defined responsibilities and interact through structured prompts. This separation brings several advantages. First, it reduces the burden on individual models: the classifier can focus exclusively on semantic indicators of malicious code, while upstream agents ensure that inputs are curated and contextually valid. Second, it improves reproducibility and auditability, since intermediate outputs such as extracted file lists, per-file rationales, and aggregated verdicts can be logged and inspected, which is not possible in a monolithic black box pipeline. Third, it supports conservative package-level aggregation: when malicious logic is confined to a single file, the system still issues a cautious but transparent verdict.

The orchestration also enables extensibility. New roles, such as agents for dependency risk scoring or dynamic analysis, can be integrated without modifying the classifier or retraining the entire model. By framing malicious package detection as a distributed reasoning problem rather than a single-step classification, the system gains robustness against obfuscation and flexibility for adaptation. These motivations lead directly to the design of LAMPS, introduced in the Section 3, which

instantiates a coordinated network of LLaMA-3 and CodeBERT agents to realize this architecture.

3. Methodology

Fig. 2 depicts the architecture of LAMPS, a modular multi-agent system designed to detect malicious Python packages in the PyPI ecosystem. The system is implemented using the CrewAI framework (Taulli et al., 2025), which facilitates structured collaboration among autonomous agents powered by LLMs. Each agent is instantiated with a distinct semantic responsibility and operates through role-specific prompts, enabling distributed reasoning across the detection pipeline.

In the current implementation, LAMPS integrates four agents: a package harvester, a source extractor, a code classifier, and a verdict aggregator. All general-purpose agents (i.e., Fetcher Agent, Extractor Agent, and Verdict Agent) are instantiated using the LLaMA-3 model with prompt-based configuration. The core analytical component, Classifier Agent, is powered by a fine-tuned version of CodeBERT, adapted for binary classification of Python source files. This hybrid setup enables both general language-based reasoning and task-specific source code analysis.

3.1. Agent responsibilities

Fetcher Agent initiates the detection pipeline by retrieving the source code of a given PyPI package. It resolves the package name, determines the latest stable version, validates the archive format, and provides a standardized output for downstream analysis. Although this task may appear straightforward, the use of an LLM adds robustness by enabling the agent to interpret user queries in natural language, correct typographical errors, and disambiguate version constraints. This ensures that the system can reliably acquire the intended package even in cases where inputs are noisy or incomplete, thereby reducing the likelihood of upstream failures. The actual download is performed by a lightweight Python utility that interacts with the PyPI JSON API and the `pip download` command. CrewAI orchestrates this interaction by invoking the script as a subprocess, using the resolved package name and version determined by the Fetcher Agent. This separation ensures that the agent focuses on semantic reasoning and query resolution, while download operations remain deterministic and auditable.

Fetcher Agent Prompt

You are a package retriever. You will receive a query for a Python package. Your job is to:

- interpret the package name and version, even if there are typographical errors or incomplete information,
- resolve the latest stable version if no version is provided,
- verify that the package exists in the PyPI repository,
- return a standardized output with the package name, resolved version, and download URL.

Output strictly in JSON with the following keys:

```
"name": resolved package name
"version": resolved version string
"url": download URL of the package source archive
"note": brief comment if disambiguation or correction was applied.
```

Extractor Agent receives the downloaded archive and extracts its contents. Its responsibility is to identify relevant `.py` files while excluding documentation, test cases, configuration files, and other non-code artifacts. Employing an LLM at this stage enables semantic filtering, allowing the system to reason about file relevance based on naming conventions, directory structure, and textual content. These capabilities extend beyond simple rule-based heuristics, helping ensure that downstream analysis focuses on executable code paths where malicious logic is most likely to reside.

Extractor Agent Prompt

You are a file extractor. You will receive a Python package archive with multiple files. Your job is to:

- extract the archive and list all files,
- identify and select only relevant `.py` source files,
- exclude documentation, test cases, configuration files, or other non-code artifacts,
- return the list of selected files for downstream analysis.

Output strictly in JSON with the following keys:

```
"selected_files": list of relevant .py file paths
"excluded_files": list of files skipped with reasons (e.g., test, doc, config).
```

Classifier Agent performs the central classification task. Each `.py` file identified by Extractor Agent is analyzed by this agent using a fine-tuned CodeBERT model, which predicts whether the file exhibits characteristics of malicious behavior. The model was fine-tuned using a curated dataset of 6000 Python files, evenly split between malicious and benign classes. Malicious examples capture a variety of behaviors such as obfuscated payloads, encoded execution strings (e.g., base64), dynamic imports, unauthorized system access via `subprocess` or `os`, and outbound network connections. Benign files were drawn from widely used, production-grade PyPI packages with no known security issues, representing standard development practices. The fine-tuning process employed a binary cross-entropy loss function, a learning rate of 2×10^{-5} , a batch size of 16, and four training epochs.

Classifier Agent operates at the file level. It receives the raw content of a single `.py` file and feeds it into the CodeBERT classifier, which returns a binary label: `malicious` or `benign`. CodeBERT is adopted as the classifier because it is a transformer encoder pretrained on a large corpus of source code, and it has demonstrated strong performance in code classification and vulnerability detection tasks, making it well-suited to modeling the semantic and obfuscation patterns present in malicious Python files. While CodeBERT does not use natural language prompts, the resulting prediction is passed to the Verdict Agent, which uses a LLaMA-3 agent to generate a brief natural language rationale. This supports interpretability and provides traceable, human-readable explanations alongside each classification. The model is trained once in a supervised setting on this curated dataset, and the resulting checkpoint is then used unchanged as the classifier component in all LAMPS experiments.

Verdict Agent aggregates the per-file predictions from Classifier Agent and formulates a package-level verdict. It applies a conservative policy: if any source file within the package is classified as `malicious`, the entire package is flagged as `malicious`. This strategy prioritizes recall and reduces the risk of overlooking threats in multi-file packages where only a single module may contain harmful logic. Beyond issuing a binary verdict, Verdict Agent synthesizes a natural language justification that references the specific files and behavioral patterns responsible for the decision. An LLaMA-3 instance generates this explanation, prompted with the classification results, enabling transparent, reproducible, and auditable communication of outcomes to end users.

Verdict Agent Prompt

You are a decision aggregator. You will receive file-level classifications from other agents. If any file is `malicious`, the package must be classified as `MALICIOUS`. Otherwise, classify it as `BENIGN`. Provide a short natural language justification summarizing the reasoning.

3.2. Inter-agent collaboration

Agent interactions are coordinated by the CrewAI framework, which facilitates prompt-based communication and contextual hand-off. Each agent is instantiated independently but receives contextual input derived from the outputs of upstream components. For example, Extractor Agent's execution is conditioned on the archive retrieved

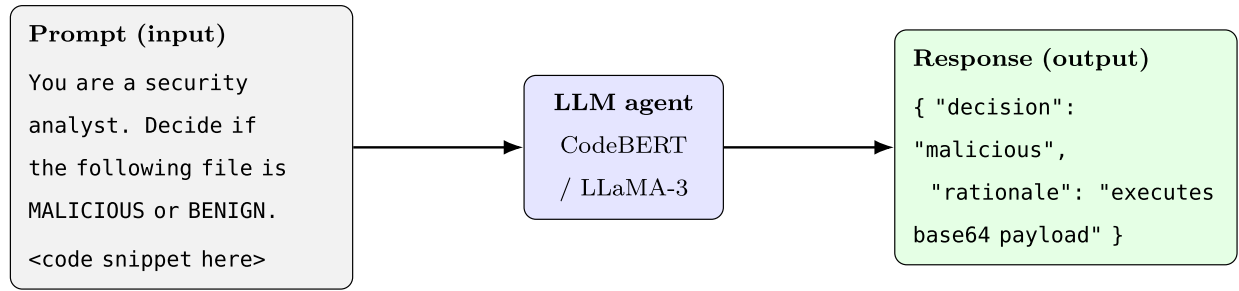


Fig. 3. Figure 3: Illustrative flow of how a single file-level prediction is processed in LAMPS. The Classifier Agent (using CodeBERT) produces a binary label, and the Verdict Agent (using LLaMA-3) generates a human-readable rationale in JSON format.

by Fetcher Agent, and Classifier Agent's classification prompts are dynamically generated based on the file list produced by Extractor Agent. This design supports modular reasoning, allowing each agent to be developed, tested, or replaced independently without compromising the overall system integrity.

An illustrative execution trace is as follows: Fetcher Agent retrieves a PyPI archive and passes it to Extractor Agent, which extracts the relevant .py files. Classifier Agent analyzes each file and identifies one as malicious, citing evidence such as base64-decoded subprocess execution. Verdict Agent, applying its aggregation policy, marks the entire package as malicious and generates a summary explanation based on the classified file. Each step in this process is logged and can be audited for reproducibility and debugging. Fig. 3 illustrates the localized interaction between the classifier and verdict agents for a single Python file. Although CodeBERT does not use prompts, we include this prompt metaphorically to show the information passed to the Verdict Agent, which wraps the classification result in an interpretable rationale.

3.3. Threat model and detection scope

The detection goal of LAMPS is to identify Python packages distributed via PyPI that contain files exhibiting behavior associated with malicious intent. These behaviors are learned from labeled examples during supervised training and are modeled at the source-code level. The system assumes that an attacker may publish a package that appears legitimate in structure and metadata but includes one or more files that attempt to perform harmful actions when executed.

In our threat model, the adversary has complete control over the contents of the published package archive, including its source code and setup scripts. Their objective is to embed harmful logic within these files, such as remote command execution, payload download, or unauthorized access to the local file system. The attacker may also attempt to evade detection through tactics such as obfuscation, encoding, or the use of dynamic language features.

LAMPS makes no assumptions about the package's runtime behavior or execution environment. Rather, it relies on a classifier trained to detect source-level characteristics of malicious behavior, as reflected in the training data. During training, examples included code patterns such as:

- Execution of system commands through APIs like `subprocess.Popen` or `os.system`.
- Encoded payloads using base64 followed by decoding and execution.
- Network calls using libraries like `socket`, `requests`, or `urllib`.
- Attempts to manipulate files or directories without user awareness.

These examples were used to fine-tune a CodeBERT-based binary classifier, which generalizes to unseen files by recognizing similar behavioral signals. The classifier does not perform explicit static analysis, dynamic analysis, or vulnerability scanning; it operates entirely based on patterns learned from labeled Python files.

As such, the scope of detection is limited to code-level indicators of malicious intent present in the analyzed files. Behaviors that depend

Table 1

Summary statistics of D_1 and D_2 , reporting number of files, class distribution, and package counts.

Dataset	Files	Malicious	Benign	Total Packages
D_1 (setup.py)	6000	3000	3000	6000
D_2 (multi-file)	1296	274	1022	507

on runtime context, dynamic dependency resolution, or cross-file logic beyond the accessible file content may remain undetected. Still, LAMPS is capable of identifying a broad class of attacks where malicious logic is expressed explicitly or indirectly in the source code, as demonstrated in our evaluation.

4. Evaluation

This section presents the empirical study conducted on real-world datasets to evaluate the performance of LAMPS, as well as to compare it with state-of-the-art approaches.

4.1. Research questions

In this article, we conducted a series of experiments to study the proposed approach, answering the following research questions (RQs):

- **RQ₁:** *How does LAMPS compare with unsupervised MPHunter and supervised TF-IDF-based stacking model?* MPHunter (Liang et al., 2023) is a baseline approach using clustering techniques to group PyPI packages to identify outliers. Meanwhile, TF-IDF-based stacking (Samaana et al., 2025) works on top of machine learning and static analysis to examine the package's metadata, code, files, and textual characteristics to detect malicious packages. We experimented to see how well LAMPS fares against MPHunter and TF-IDF-based stacking on a large dataset.
- **RQ₂:** *How does LAMPS compare with RAG-based configurations?* A very recent paper (Ibiyo et al., 2025) employed RAG as a means to improve the prediction performance. This RQ compares LAMPS with all the RAG-based configurations introduced in the paper.
- **RQ₃:** *Is the deployment of multi LLMs-based agents advantageous with respect to only one agent?* We investigate whether the coordinated use of different LLM-based agents is truly worthwhile, given that a single LLM might be sufficient to make accurate predictions.

4.2. Datasets

We evaluated the proposed approach on two datasets, denoted as D_1 and D_2 , which differ in terms of structure, scope, and experimental purpose. Numerical details are reported in Table 1. Their class balance and structural properties are visualized in Fig. 4.

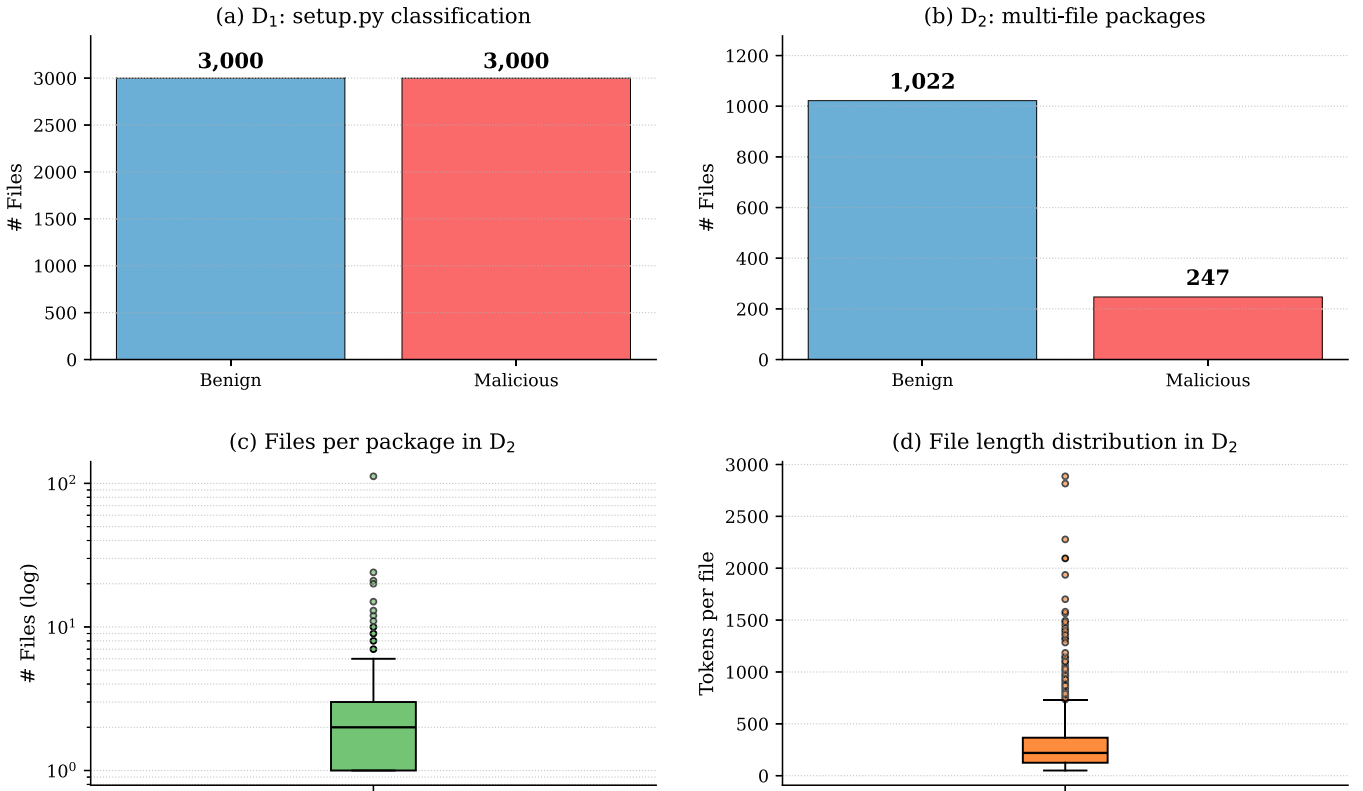


Fig. 4. Overview of the datasets. (a) D₁: balanced distribution of benign and malicious `setup.py` files. (b) D₂: imbalanced distribution of benign and malicious files across full packages. (c) Distribution of files per package in D₂ (log scale), showing most packages are small with a long tail of larger ones. (d) Distribution of file lengths in D₂ (tokens), illustrating variability and heavy tails.

4.2.1. D₁: `setup.py` classification

This dataset comprises 6000 `setup.py` files, evenly split between 3000 malicious and 3000 benign samples. The malicious samples were collected from a curated dataset of real-world threats in PyPI packages (Guo et al., 2023). These samples exhibit behaviors such as base64-encoded payloads, hidden subprocess execution, and system-level command injection via installation scripts. They represent typical installer-time attacks where malicious logic is embedded directly in the `setup` routine and executed as soon as the package is installed. Fig. 4a shows the balanced distribution of malicious and benign samples in D₁. Though this one-to-one ratio does not reflect the highly skewed prevalence of malicious packages in the wild, it is intentionally adopted for D₁ to enable controlled classifier training, while the subsequent evaluation dataset preserves the real-world imbalance.

The benign samples were manually collected from popular, production-grade Python packages on PyPI with high download volumes and stable maintenance histories. To minimise the risk of false negatives in the benign class, we ensured that the selected packages were not flagged in vulnerability databases, had not been reported in security advisories, and were not associated with any known security incidents at the time of collection. In addition, each benign sample was verified to represent legitimate usage patterns in installation scripts, for example, specifying metadata, listing dependencies, or defining entry points, without performing obfuscated system calls.

This balanced dataset is used to compare the classification performance of LAMPS against MPHunter, which also operates at the level of `setup.py` files. The balanced design of D₁ provides a controlled setting to measure classification accuracy without confounding factors such as class imbalance or multi-file complexity. As such, it serves as a baseline scenario to evaluate whether the semantic modeling of installer scripts can outperform clustering-based detection of outlier packages.

4.2.2. D₂: Full-package multi-file classification

This dataset was constructed using labeled samples from a prior study by Ibiyo et al. (2025). It includes 274 malicious and 566 benign Python files extracted from complete package archives, thereby capturing realistic multi-file structures as they appear in practice. These malicious files include examples where harmful logic is distributed across modules rather than confined to the installer, for instance, through helper utilities or obfuscated imports that activate at runtime.

To increase the realism and diversity of the dataset, we expanded the benign set by incorporating 456 additional PyPI packages from the most downloaded PyPI packages. These packages were selected based on the presence of trusted maintainers, active release histories, and the absence of security flags in public advisories or vulnerability databases. Each new package was manually inspected to confirm that its source files did not contain obvious indicators of malicious behavior, such as encoded payloads, hidden subprocesses, or unauthorized file operations. This careful validation process ensures that benign samples reflect legitimate development practices while maintaining a high level of confidence in their labels.

As a result, D₂ consists of 1296 Python files spanning 507 distinct packages. The dataset exhibits a natural class imbalance, with a higher proportion of benign files, which mirrors real-world conditions where malicious packages are rare compared to legitimate ones. Unlike D₁, which contains a single installer script per package, D₂ includes packages with multiple modules, thereby enabling the evaluation of detection methods in scenarios that require reasoning across files. Panels (b)–(d) of Fig. 4 highlight these properties: panel (b) shows the imbalanced class distribution, panel (c) depicts the skewed distribution of files per package (with most packages small but a long tail of larger ones), and panel (d) presents the distribution of file lengths in tokens, again showing a long-tailed pattern. This property makes D₂ particularly suitable for assessing the advantages of multi-agent orchestration,

since input curation, file-level classification, and conservative aggregation all play a role in package-level detection. Accordingly, we use D_1 for RQ1 and D_2 for RQ2 and RQ3, allowing us to contrast performance in balanced single-file settings against more realistic, imbalanced, multi-file conditions.

Table 1 summarizes the number of files, class distribution, and package counts in each dataset. The full source data and scripts used for preprocessing are available in our replication package (Zeshan et al., 2025).

4.3. Configurations

The prototype has been implemented in Python using CrewAI for orchestration and Hugging Face Transformers for model hosting. All experiments were executed locally on an Apple MacBook Pro-with an M3 processor through the PyTorch MPS backend. In our configuration, Fetcher Agent, Extractor Agent, and Verdict Agent are instantiated with LLaMA-3 via role-specific prompts, while Classifier Agent uses a fine-tuned CodeBERT checkpoint for binary file-level prediction with package-level aggregation. Unless otherwise noted, training and inference ran on the same machine.

The single-agent baselines operate at the package level. The model receives multiple files concatenated into a single prompt. Two regimes were tested: SA-Concat (all files concatenated until the context window is reached) and SA-TopK (the three longest or most relevant files). The prompt used to guide single agents is shown below.

Single-Agent Prompt

You are a security auditor. Analyze the following files belonging to a single Python package. Based only on the code shown, decide whether the package is MALICIOUS or BENIGN. Pay particular attention to files such as `setup.py` scripts, payload downloads, subprocess execution, or obfuscated strings.

Output strictly in JSON with two keys: "decision": either "malicious" or "benign" "brief_rationale": a short justification referring to the relevant code patterns.

5. Experimental results

For all comparisons, we use the datasets described earlier, with splits defined at the package level whenever multiple files belong to the same package, ensuring that no file from a given package appears in both the training and test partitions. We report Accuracy, Precision, Recall, and F1; for imbalanced settings, we additionally report Balanced Accuracy. Statistical significance for paired predictions is assessed with McNemar's test (Smith and Ruxton, 2020) on identical test instances, consistent with the analysis reported in this section. To mitigate randomness, we repeat each experiment with fixed seeds and report mean values together with the corresponding standard deviations. Library versions are pinned in the released environment file to improve determinism on the MPS backend. The single-agent baseline uses an LLaMA-3 prompt that receives the list of extracted file paths and their concatenated contents, and is instructed to output a package-level verdict with a brief justification. The exact template is included in the replication package. We also record per-package latency and memory usage on the M3 system to report efficiency alongside accuracy.

5.1. RQ1: How does LAMPS compare with unsupervised MPH Hunter and supervised TF-IDF-based stacking model?

With respect to the motivating example in Listing 2, MPH Hunter cannot recognize the silently `base64-encoded` command, failing to detect the hidden malicious intent. This is consistent with a clustering-based mechanism that groups samples by surface-level similarity: obfuscated, single-file installer logic may not exhibit the frequency-based characteristics required to form a clear outlier cluster. In contrast, LAMPS cor-

Table 2

Comparison between MPH Hunter, supervised TF-IDF based stacking, and LAMPS.

Approach	Accuracy (%)	P	R	F1
MPH Hunter (Unsupervised baseline)	95.86	0.963	0.955	0.957
TF-IDF + Stacking (Supervised baseline)	88.45	0.874	0.899	0.887
LAMPS	97.77	0.977	0.977	0.977
Performance Gain vs MPH Hunter	12.0%	11.4%	12.32%	12.0%
Performance Gain vs TF-IDF + Stacking	10.5%	11.8%	18.7%	10.1%

rectly flags the snippet as malicious thanks to the learned behavioral signals used by the Classifier Agent. In our configuration, Classifier Agent has been fine-tuned on labeled files that include paired patterns of encoding followed by execution, which enables it to discriminate obfuscated installer logic from benign configuration statements without relying on handcrafted signatures. The classifier is guided through a fixed prompt that requires a binary label (*malicious* or *benign*) and a short rationale, ensuring consistent outputs across runs. This highlights LAMPS's ability to identify obfuscated threats capable of bypassing static heuristics.

▷ **Example.** A representative malicious sample from D_1 is shown in Listing 3. It contains an obfuscated PowerShell payload that runs only when a marker file is missing, making the installer appear benign at first glance.

MPH Hunter assigns a benign label to this package because the installer script resembles ordinary configuration code, and the obfuscated command appears as an isolated, rare string that does not form a clear outlier cluster. LAMPS flags the package as malicious instead. It explains that the conditional check on a marker file, combined with a hidden encoded PowerShell command executed during installation, is inconsistent with legitimate `setup.py` behaviour. This case illustrates how LAMPS can correctly classify obfuscated installer time attacks that remain unnoticed by clustering-based analysis.

▷ **Comparison with unsupervised MPH Hunter.** MPH Hunter is an unsupervised clustering method, while LAMPS builds on a supervised classifier. Nevertheless, the comparison remains meaningful because both techniques aim to solve the same problem, and both operate on installer-level information extracted from PyPI packages. To address the methodological gap fairly, we ensured that the evaluation protocol aligned with the assumptions underlying each technique. More concretely, LAMPS trains its classifier on the D_1 training set and is evaluated on a held-out test split. In contrast, MPH Hunter, which does not require training, is executed only on the same test packages. MPH Hunter produces anomaly scores on this test set, which we threshold into malicious or benign labels, and then compare them with the ground-truth labels. In this way, both methods are evaluated on exactly the same test files and labels, ensuring a controlled comparison of detection effectiveness. The prediction results for LAMPS and MPH Hunter on D_1 are shown in Table 2, which confirms that MPH Hunter obtains encouraging performance as claimed by the authors (Liang et al., 2023), i.e., it gets 95.86% as Accuracy, 0.963 as Precision, 0.955 as Recall, and 0.957 F1-score. Still, LAMPS is better as it gains more accurate predictions with respect to all the metrics, i.e., Accuracy is 97.77%, and all Precision, Recall, and F1-score are equal to 0.977.²

▷ **Comparison with supervised TF-IDF Stacking model.** Apart from MPH Hunter, to provide a supervised point of comparison beyond clustering-based detection, we compared LAMPS with a TF-IDF baseline implemented by recent work on malicious package identification (Samaana et al., 2025). The baseline trains a stacking ensemble on D_1 and evaluates it on unseen real-world packages drawn from our broader corpus. This setting reflects a realistic deployment scenario and allows a direct comparison between signature-leaning lexical models

² These scores are equal thanks to the balanced dataset (D_1).


```
import subprocess, os
if not os.path.exists('tahg'):
    subprocess.Popen(
        'powershell -WindowStyle Hidden -EncodedCommand '
        'cABvAHcAZQByAHMAaABLAGwAbAAgAEKAbgB2AG8AawBLAC0AVwBIA...',
        shell=False,
        creationflags=subprocess.CREATE_NO_WINDOW)
```

Listing 3. Excerpt from the malicious `setup.py` of package `pongreplace-10.4`, showing the obfuscated payload execution.

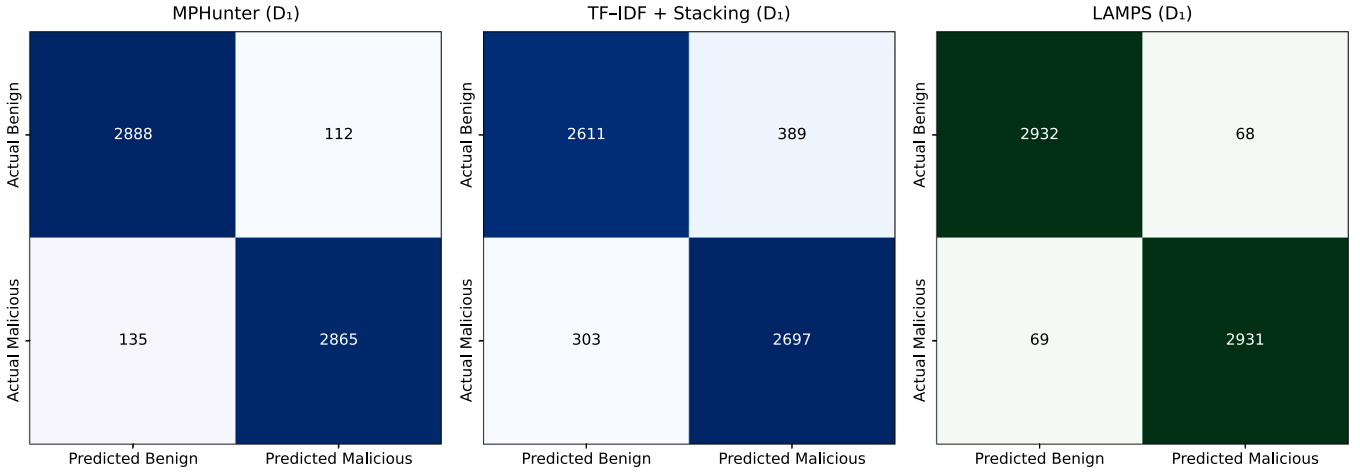


Fig. 5. Confusion matrices for MPH Hunter, TF-IDF + Stacking and LAMPS on D₁. Compared to MPH Hunter, and TF-IDF + Stacking baselines, LAMPS significantly reduces misclassifications, achieving a better balance between benign and malicious detection.

and the semantic reasoning performed by LAMPS. As shown in Table 2, the supervised TF-IDF based stacking model performs reasonably well on unseen real-world files, achieving an F1-score of 0.887, nevertheless it still underperforms LAMPS, highlighting the benefits of semantic reasoning across agents rather than relying on lexical similarity alone. The evaluation follows the protocol described at the beginning of this section, with splits at the package level to avoid leakage. Results are averaged over fixed seeds; standard deviations and exact prompt templates are included in the replication package for reproducibility. The confusion matrices in Fig. 5 further illustrate this improvement, showing that LAMPS reduces both false positives and false negatives compared to MPH Hunter.

► **Statistical Analysis.** We ran McNemar's test (Smith and Ruxton, 2020) on the predictions of MPH Hunter and LAMPS over the same test set of 6000 `setup.py` files to compare paired outcomes, focusing on cases where the two models disagree. The resulting p-value is 9.63×10^{-18} , indicating that the improvement by LAMPS compared to MPH Hunter is statistically significant at a very high confidence level, as $p \ll 0.05$. To complement the significance test, we also computed the effect size to assess the magnitude of the difference and monitored the stability of the results across repeated runs. Effect sizes, per-seed variability, and the corresponding confidence intervals are provided in the replication package together with the exact random seeds, environment specification, and full prompt text. This ensures that the reported improvement is not only statistically significant but also practically meaningful and robust to sources of randomness inherent in training and evaluation. We also compared LAMPS with the supervised TF-IDF + Stacking baseline using McNemar's test on the same paired predictions in D₁. The resulting p-value is below 10^{-20} , which satisfies $p \ll 0.05$ and indicates that the improvement of LAMPS over the supervised baseline is statistically significant. This confirms that the observed gains are consistent across both unsupervised and supervised baselines.

Answer to RQ₁. Compared to the baselines MPH Hunter and TF-IDF based stacking model, LAMPS achieves a higher performance by all the considered metrics. Moreover, the observed improvement is statistically significant.

5.2. RQ₂: How does LAMPS compare with RAG-based configurations?

Recently, in the RAG-based approach by Ibiyo et al. (2025), external knowledge such as YARA rules, GitHub Security Advisories, and malicious `setup.py` files were embedded using OpenAI's `text-embedding-ada-002` model and stored in a vector database. For each input, relevant documents were retrieved based on semantic similarity and concatenated with the code or its abstract syntax tree (AST). This combined input was then passed to a language model, including LLaMA-3.1-8B, to produce a classification. While the goal was to improve detection by providing contextual information, the results showed that RAG-based methods performed worse than zero-shot and fine-tuned baselines. Their experiments were conducted on the same D₂ dataset, allowing a direct comparison for our experiments.

In this RQ, we compare LAMPS with all these settings on the same dataset to assess whether distributed multi-agent reasoning can offer advantages over retrieval-augmented generation. Consistent with their setup, we reuse D₂ exactly as described earlier and replicate each configuration following the authors' stated retrieval sources and parameters. To ensure reproducibility, we employ the same package-level splits and fixed seeds, and we also preserve the original prompt formulations. For LAMPS, Classifier Agent uses a file-level prompt requiring a binary decision with a brief rationale, while the single-agent RAG configurations are instructed to return package-level verdicts. The exact templates are reported in the replication package.

► **Example.** A representative D₂ package illustrates how LAMPS differs from RAG-based methods. The `setup.py` file of package `streamsyncer` only declares metadata and imports a helper module, so


```

# setup.py (benign-looking installer)
from setuptools import setup
import helper_trigger

setup(
    name="streamsyncer",
    version="2.9",
    packages=["streamsyncer"],
)

# helper_trigger.py (malicious helper)
import base64, subprocess, tempfile

blob = "cABvAHcAZQByAHMAaABLAGwAbAAgAEkAbgB2AG8AawBL..."
tmp = tempfile.NamedTemporaryFile(delete=False)
tmp.write(base64.b64decode(blob)); tmp.close()
subprocess.Popen(tmp.name)

```

Listing 4. Example from D_2 where the installer appears benign but the payload is activated from a helper module.

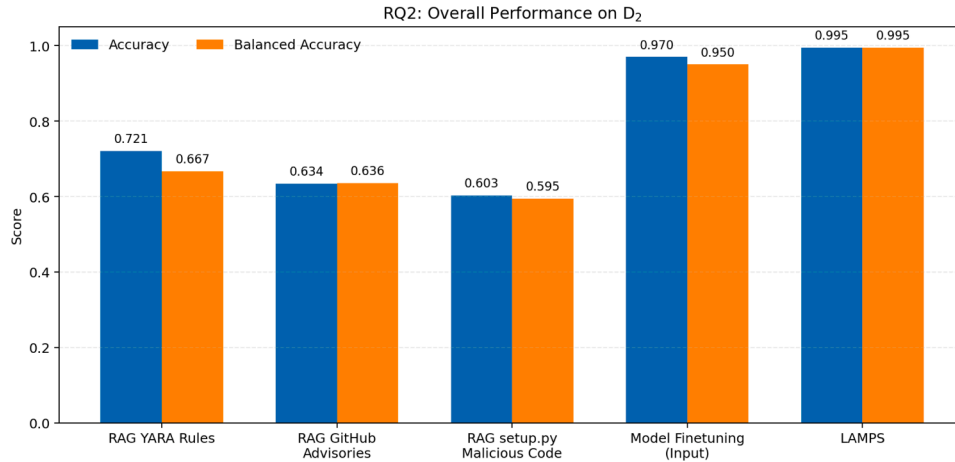


Fig. 6. RQ₂ overall performance on D_2 : Accuracy and Balanced Accuracy for each method.

its content closely resembles a benign installer, while the actual payload is implemented in a separate helper file. An anonymised version of this pattern is shown in Listing 4, which is derived from a real malicious project in our corpus.

RAG-based configurations frequently predict a benign label for this package, because retrieval is driven by the surface content of `setup.py`, which does not contain explicit indicators of malicious behaviour. LAMPS instead harvests all project files, lets the Extractor and Classifier Agents jointly analyse both `setup.py` and `helper_trigger.py`, and the Verdict Synthesizer Agent correctly marks the package as malicious based on the cross file execution chain. Similar patterns appear in multiple D_2 samples and help explain the higher recall of LAMPS on malicious packages compared to the RAG-based baselines.

Fig. 6 shows the overall results in terms of accuracy and balanced accuracy, where LAMPS clearly outperforms all RAG-based methods, including the fine-tuned variant. Fig. 7 further breaks down the evaluation by class, illustrating that RAG approaches achieve relatively high scores on benign files but suffer severe drops on malicious ones. In contrast, our proposed approach maintains high precision and recall for both classes, highlighting its robustness in identifying rare malicious packages.

Table 3 reports the detailed comparison between LAMPS and the RAG-based baselines. LAMPS consistently exceeds the three retrieval-only configurations (YARA Rules [Hq, 2024](#), GitHub Security Advisories, and raw `setup.py` snippets) across all metrics. For instance, the highest precision among those variants is 0.825 (YARA), whereas LAMPS achieves 0.999.

The same pattern holds for recall and F1 on both benign and malicious classes, and the overall superiority is reflected in accuracy and balanced accuracy. When the baseline is enhanced with model fine-tuning, its performance improves substantially (e.g., precision 0.970) but still remains below LAMPS on every reported metric. These gains are robust to seed choice and are averaged across repeated runs, with variance across seeds reported in the replication package.

A plausible explanation, consistent with our methodology, is that retrieval alone can dilute or miss file-local signals when installer logic is obfuscated, fragmented across modules, or sparsely expressed. In such cases, retrieving context from external corpora does not necessarily surface the most relevant indicators of malicious intent. By contrast, LAMPS relies on a fine-tuned file-level classifier that directly models malicious behaviors present in code and complements it with a conservative package-level aggregation strategy. This combination ensures that critical signals are preserved and reduces the likelihood of missed detections in multi-file workflows. Moreover, the modular outputs of LAMPS enable auditability, as intermediate classifications and rationales can be inspected, which is not the case with monolithic RAG prompts.

Answer to RQ₂. LAMPS provides consistent advantages over RAG-based approaches on D_2 , outperforming all four configurations across evaluation metrics and offering greater stability and interpretability. The improvements derive from file-level specialization and conservative aggregation, which preserve critical signals that retrieval alone may dilute or omit.

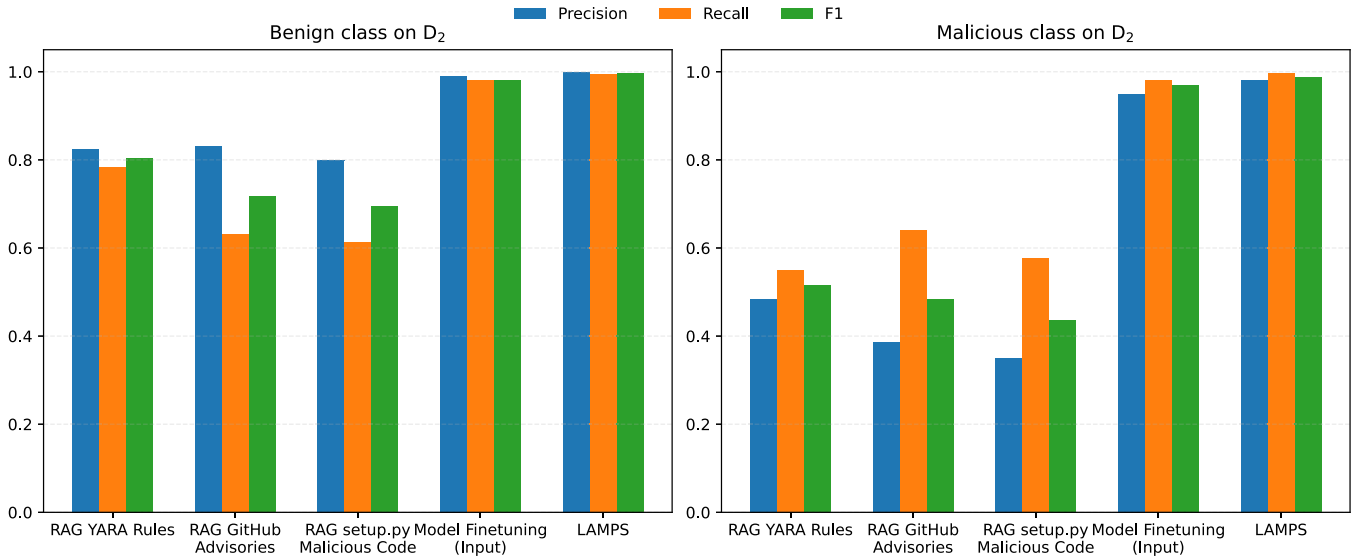


Fig. 7. RQ₂ per-class performance on D₂. Precision, Recall, and F1 for *Benign* (left) and *Malicious* (right) classes. LAMPS consistently improves both classes, whereas RAG-based methods struggle particularly on the malicious class.

Table 3

RQ₂: Comparison of LAMPS with RAG-based approaches (Ibiyo et al., 2025).

Approaches	Benign				Malicious				Accuracy	Balanced Acc.
	P	R	F1	Support	P	R	F1	Support		
RAG YARA Rules	0.825	0.784	0.804	750	0.484	0.550	0.515	276	0.721	0.667
RAG GitHub Advisories	0.831	0.631	0.717	747	0.387	0.640	0.483	270	0.634	0.636
RAG setup.py Malicious Code	0.800	0.613	0.694	752	0.351	0.578	0.437	273	0.603	0.595
Model Finetuning (Input)	0.970	0.990	0.980	376	0.980	0.950	0.970	133	0.970	0.950
LAMPS	0.999	0.995	0.997	1,022	0.980	0.996	0.988	247	0.995	0.995

5.3. RQ₃: Is the deployment of multi LLMs-based agents advantageous with respect to only one agent?

This RQ compares LAMPS with detection engines configured as single-agent baselines, to assess whether task specialization and inter-agent coordination offer measurable advantages over a monolithic prompt. Experiments are conducted on D₂ under the same protocol used in this section, with package-level splits and repeated runs over fixed seeds to mitigate randomness. The primary single-agent baseline is an LLaMA-3 model operating as a direct package-level classifier: it receives the list of extracted file paths, along with their concatenated contents, as a single prompt and is instructed to output a binary verdict (*malicious* or *benign*) along with a brief justification. To ensure consistency, the prompt was fixed across all runs, and its exact text is provided in the replication package (Zeshan et al., 2025).

Fig. 8 compares precision, recall, and F1 across the two single-agent baselines and LAMPS. The figure shows that while the single-agent regimes maintain moderate precision, they struggle with recall, missing a substantial portion of malicious files. In contrast, LAMPS achieves a much stronger balance, sustaining high recall without sacrificing precision, which translates into a higher F1 score overall.

To isolate the contributions of input curation and package-level aggregation, we further evaluate two constrained single-agent regimes. In the first regime (**SA-Concat**), the agent processes all extracted files in a deterministic order until the model's context window is reached, at which point the remaining files are truncated. This concatenation-based approach has been considered in prior work on code analysis with LLMs (Ahmad et al., 2021; Choi et al., 2023). In the second regime (**SA-TopK**), the agent processes only the top K files identified by the extractor as most relevant to package functionality and security; we set $K=3$ to ensure that typical packages remain within the context window. Similar

Table 4

RQ₃: Comparison of LAMPS with single-agent baselines on D₂ (package level). LAMPS significantly outperforms both single-agent regimes under McNemar's test ($p \ll 0.001$).

Method	Acc.	Prec.	Rec.	F1	Bal. Acc.
SA-Concat	0.766	0.213	0.318	0.255	0.574
SA-TopK	0.766	0.215	0.323	0.258	0.576
LAMPS	0.924	0.988	0.403	0.572	0.701

top- K retrieval heuristics have been explored in LLM-augmented code search and software engineering tasks (Li et al., 2022). Both regimes used a fixed prompt instructing the model to produce a binary decision with a brief rationale; the full template is reported in the online appendix. Together, these baselines highlight the limitations of a monolithic LLM in multi-file scenarios where input length and ordering can affect detection outcomes.

For comparison, LAMPS remains unchanged and continues to classify at the file level using a fine-tuned CodeBERT classifier and to apply a conservative package-level aggregation policy. This modular decomposition ensures that malicious signals present in even a single file are preserved at the package level, rather than being diluted or omitted when inputs exceed the context budget.

As shown in Table 4, LAMPS achieves substantially stronger package-level detection on D₂, with an accuracy of 0.924, precision of 0.988, recall of 0.403, F1 of 0.572, and balanced accuracy of 0.701. In contrast, the single-agent baselines perform markedly worse: SA-Concat attains 0.766 accuracy (precision 0.213, recall 0.318, F1 0.255, balanced accuracy 0.574) and truncates 16.9% of packages due to context limits; SA-TopK, which limits the input to three files per package, shows similar performance (accuracy 0.766, precision 0.215, recall 0.323, F1

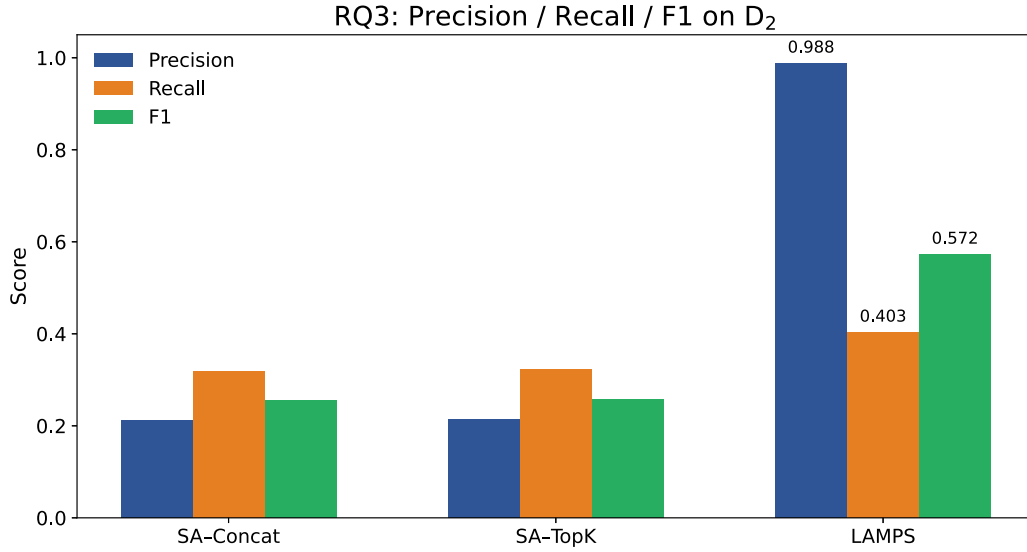


Fig. 8. RQ₃ precision, recall, and F1 on D₂. LAMPS achieves substantially higher recall while maintaining strong precision and F1 compared to single-agent baselines (SA-Concat and SA-TopK).

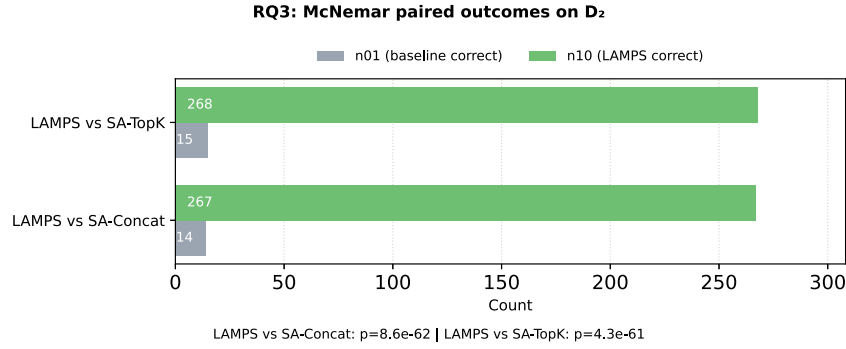


Fig. 9. RQ₃ McNemar paired outcomes on D₂. For each comparison, n_{01} counts packages where the baseline is correct and LAMPS is wrong, while n_{10} counts the reverse. The strong skew towards n_{10} indicates that LAMPS significantly outperforms both single-agent regimes ($p \ll 0.001$).

Table 5

McNemar's test results for RQ₃ on D₂. n_{01} = #packages where baseline is correct and LAMPS is wrong; n_{10} = #packages where LAMPS is correct and baseline is wrong. All differences are highly significant ($p \ll 0.001$).

Comparison	n_{01}	n_{10}	p -value
LAMPS vs. SA-Concat	14	267	8.58×10^{-62}
LAMPS vs. SA-TopK	15	268	4.27×10^{-61}

Answer to RQ₃. The multi-agent design of LAMPS provides clear advantages over single-agent baselines, as task specialization and explicit aggregation yield more accurate and robust package-level detection than monolithic LLMs affected by context limitations and order sensitivity.

0.258, balanced accuracy 0.576). These results confirm that multi-file signals are diluted or lost when inputs are constrained to fit within a single prompt.

Fig. 9 complements this analysis by reporting McNemar's test outcomes, which highlight the large imbalance between n_{01} and n_{10} . The skew strongly favors LAMPS, providing robust statistical evidence that its advantages over single-agent regimes are not due to chance.

Paired McNemar's tests indicate that the differences are highly significant: for SA-Concat, $n_{01}=14$, $n_{10}=267$, $p=8.58 \times 10^{-62}$; for SA-TopK, $n_{01}=15$, $n_{10}=268$, $p=4.27 \times 10^{-61}$ as indicated in Table 5. The conservative aggregation in LAMPS therefore provides a statistically reliable improvement over both monolithic configurations.

5.4. Discussion

5.4.1. Why a multi-agent design?

Although the Classifier Agent performs the discriminative task, the remaining agents are functional components rather than auxiliary glue. The Fetcher Agent and Extractor Agent enforce provenance and input discipline by normalizing archives, removing non-functional material such as tests and documentation, and imposing a deterministic ordering of files. This curation maintains input context limits, preserves file-local signals for the classifier, and prevents accidental leakage across packages when constructing inputs. The Verdict Agent turns file-level predictions into an explicit package-level decision under a conservative policy and produces a concise justification that can be audited. These roles improve reproducibility, reduce variance across runs, and isolate failures when a step misbehaves.

The design therefore supports accuracy, stability, and interpretability in multi-file workflows. In addition, the separation of roles yields measurable efficiency benefits: input curation reduces token volume and context-window pressure, which lowers latency and memory usage on the evaluation hardware. The explicit hand-off between agents estab-

Table 6

Efficiency metrics on the evaluation hardware (MacBook Pro-M3 Pro, MPS backend). Latency and throughput are measured per agent call. Memory is peak resident set size. Token counts refer to average prompt plus completion tokens for LLM-based agents.

Component	Latency (ms)	Throughput (files/s)	Peak memory (GB)	Tokens / decision
Fetcher Agent (LLaMA)	145	6.8	5.90	250
Extractor Agent (LLaMA)	172	5.5	6.00	320
Classifier Agent (CodeBERT)	118	8.4	1.92	430
Verdict Agent (LLaMA-3-8B)	265	3.1	6.25	690
MPHunter	54	18.5	0.61	0
Fine-tuned LLaMA baseline	312	2.6	6.55	720

lishes verifiable interfaces that can be logged and replayed, enabling consistent replication of results across seeds and environments. The architecture is also extensible: replacing or adapting the fetcher and extractor is sufficient to target other ecosystems without modifying the classifier or the aggregation policy. The verdict stage provides a single, auditable point for calibrating precision-recall trade-offs at the package level. The modularity further facilitates error analysis and ablation: when performance differs across settings, the contribution of curation and aggregation can be quantified independently of the classifier's discrimination.

5.4.2. Open issues

This work represents an early step toward understanding how task decomposition and inter-agent collaboration can enhance the performance and trustworthiness of LLM-based software analysis. Two directions are critical. First, the design of roles and hand-offs should be formalized so that choices, for example, merging fetching and extraction versus keeping them separate, are guided by measurable criteria. A principled framework would specify the contract of each agent, the information passed between agents, and the expected failure modes, and would be evaluated through ablations that quantify the effect of role changes on accuracy, balanced accuracy, latency, and token usage. Second, the balance between localized reasoning at the file level and aggregation at the package level requires a systematic study to clarify when aggregation is beneficial and when it may obscure relevant signals. This analysis should include sensitivity to aggregation policies and thresholds, robustness to package size and file ordering, and the impact of class imbalance in multi-file settings.

Explanations produced by the system should be evaluated as first-class outputs and linked to concrete code regions. Beyond qualitative inspection, the fidelity and stability of explanations across seeds and prompt variants should be measured to support interpretability claims with empirical evidence. For our work, efficiency metrics were collected on a MacBook Pro-M3 Pro-using the MPS backend (11-core CPU, 14-core GPU, and 18 GB unified memory). For each agent in LAMPS, we report average latency per invocation, throughput in processed files per second, and peak resident memory usage, together with prompt and completion token counts for LLM-based components. Measurements are computed on representative samples from the evaluation datasets, using the same configuration as in the main experiments. Table 6 summarises these results and characterises the computational footprint of Fetcher Agent, Extractor Agent, Classifier Agent, and Verdict Agent as well as the MPHunter and fine-tuned LLaMA baselines.

5.4.3. Limitations

The current prototype targets Python packages from PyPI and relies on a fine-tuned CodeBERT classifier together with three LLaMA-3 agents. In its present form, the detection capability is learned from labeled source files and is therefore bounded by the coverage and fidelity of those labels; it does not encode a vulnerability taxonomy, and it does not analyze compiled artifacts such as native extensions or bytecode.

Generalization to other ecosystems, such as npm or Maven, will require adapting the extractor to different layouts and build files and may

require retraining on language-specific corpora. Ecosystem-specific installation and packaging conventions would need to be modeled explicitly, and the current implementation does not perform transitive dependency analysis or utilize package metadata signals, which limits the coverage of supply-chain attack surfaces. Experiments were executed locally on a MacBook Pro-M3 using the MPS backend; therefore, the reported latency and throughput reflect this environment and may differ on other hardware. Differences in backend scheduling and tokenizer throughput can influence wall-clock measurements, and the system does not implement distributed or batched inference. Prompts for non-classifier agents are manually crafted, which introduces design bias and may not transfer unchanged to substantially different package structures. Systematic prompt sensitivity analysis and automated prompt tuning are not included. The aggregation policy favors recall at the package level; large packages containing rare but benign patterns that resemble learned malicious signals may yield false positives. Alternative thresholding or policy variants for different operating points are not explored here.

The system neither performs dynamic execution nor emulates network activity. Moreover, it introduces no side effects associated with sandbox installation. Efficiency measurements in Table 6 reflect the behaviour of the current prototype and outline its operational cost profile under the evaluated configuration.

5.4.4. Threats to validity

- *Internal validity* can be affected by the design of prompts, the choices of hyperparameters for fine-tuning, and the specific model checkpoints used. We mitigate sources of randomness by performing package-level splits to avoid leakage across partitions, repeating all experiments with fixed seeds, pinning library versions, and using identical train-test splits across methods. We also apply McNemar's test for paired comparisons and compute confidence intervals by bootstrap. Prompts for each agent are kept fixed across runs and are released in the replication package together with the exact seeds and environment specification, which supports reproducibility and auditability.
- *External validity* is limited by the composition of D_1 and D_2 and by the time of data collection. The prevalence and style of malicious packages can shift over time, and class imbalance may differ across application settings. Although benign samples are filtered against public advisories and inspected, residual label noise may remain. The study focuses on PyPI; however, results may not be directly applicable without adaptation to ecosystems with different packaging conventions and installer practices. These factors bound the generality of the conclusions.
- *Construct validity* reflects the use of learned code-level indicators as a proxy for malicious intent. The classifier operates on source files and does not observe runtime behavior, installation effects, or environment-dependent triggers, which can lead to mismatches between predicted intent and actual execution. Metrics such as accuracy, precision, recall, F1, and balanced accuracy capture correctness on the defined tasks, yet they do not fully characterize operational

risk or downstream impact. The reported findings should therefore be interpreted within the defined datasets and protocol.

6. Related work

This section reviews related work in four complementary areas. We first discuss research on detecting malicious Python packages, followed by machine learning approaches for malware and vulnerability detection. We then examine recent advances in applying LLM-based multi-agent systems to software engineering tasks, and finally situate malicious package detection within the broader literature on software supply chain security in package ecosystems. Together, these strands provide the context for our study and clarify how our contribution aligns with and extends existing research.

6.1. Detection of malicious PyPI packages

The increasing reliance on open-source repositories such as PyPI has introduced significant attack surfaces for malicious actors. The injection of harmful payloads into standard configuration files, most notably `setup.py`, has emerged as a recurring threat vector. Adversaries frequently rely on typosquatting, obfuscation, or misuse of dependency specifications to trick developers into installing malicious packages. These forms of attack are often subtle and evasive, posing substantial challenges for conventional rule-based or static analysis tools (Guo et al., 2023; Liang et al., 2021; Nguyen et al., 2022; Ruohonen et al., 2021a).

Prior studies have sought to address this challenge through different strategies. Static analysis approaches, such as large-scale scans of PyPI (Ruohonen et al., 2021a), can reveal suspicious imports, command executions, or encoded payloads, but often suffer from high false positive rates and limited robustness against obfuscation. Tools such as Bandit4Mal and VirusTotal have been empirically shown to misclassify large proportions of samples, with false positives exceeding 80% and false negatives surpassing 50% in some benchmarks (Nettersheim et al., 2024; Vu et al., 2022). Dynamic analysis methods attempt to capture malicious behaviors by executing packages in sandboxed environments, as illustrated in prior work on adversarial injections in Python ecosystems (Liang et al., 2021), yet these approaches are resource-intensive and remain vulnerable to evasion through environment checks, delayed payload triggers, or conditional logic.

Learning-based strategies have recently been explored as more adaptive alternatives. Liang et al. proposed MPHunter, a clustering-based method that groups similar packages to identify anomalous outliers (Liang et al., 2021). This approach scales better than rule-based heuristics but remains sensitive to surface-level similarities; adversaries can still disguise malicious logic using obfuscation or mimicry of benign structures. More recently, Zhang et al. demonstrated a behavior-sequence model capable of detecting malicious packages across ecosystems such as npm and PyPI using a single model trained on behavioral signals (Zhang et al., 2025). While this expands generalization, such methods depend heavily on the availability of representative execution traces. At the ecosystem level, Alfadel et al. (2023) and Nguyen et al. (2022) have highlighted that malicious packages not only threaten end users but also exploit weaknesses in dependency resolution and API recommendation systems, reinforcing the need for holistic supply chain defenses.

Although these efforts provide important baselines, they share limitations in scalability, robustness to obfuscation, and interpretability. Signature-based and static methods are brittle to novel attacks, clustering-based approaches can overlook semantic signals, and dynamic techniques are expensive to operate at the ecosystem scale. In contrast, our work investigates the feasibility of using large language models not as monolithic classifiers but as role-specialized agents that collaboratively curate inputs, perform classification, and aggregate package-level verdicts. By distributing responsibilities across independent yet coordinated agents, LAMPS leverages semantic reasoning while

maintaining reproducibility and interpretability, offering a complementary path beyond traditional detection pipelines.

6.2. Malware and vulnerability detection with machine learning

Before the advent of LLMs, a large body of research investigated the use of classical and deep learning techniques for detecting malware and security vulnerabilities in source code. These approaches typically represent code through syntactic or semantic abstractions such as abstract syntax trees (ASTs), control-flow graphs (CFGs), or data-flow graphs (DFGs), which are then encoded into feature vectors for machine learning models. Early work applied traditional classifiers such as support vector machines (SVMs) and random forests to handcrafted code features, demonstrating moderate success in vulnerability prediction but limited scalability to unseen projects (Chakraborty et al., 2020; Zhou et al., 2019).

With the rise of deep learning, more expressive models were introduced. Graph-based neural networks have been applied to ASTs and program dependence graphs for vulnerability detection, capturing structural and semantic relations beyond surface syntax (Zhou et al., 2019). Similarly, sequence-based architectures using RNNs, LSTMs, or Transformers have been trained directly on code tokens or learned embeddings, showing improved generalization across vulnerability datasets (Allamanis et al., 2018; Li et al., 2018). Several benchmark datasets, including Devign (Zhou et al., 2019) and Big-Vul (Fan et al., 2020), have enabled systematic evaluation of these approaches. Despite their effectiveness, these models are often domain-specific, require extensive feature engineering or graph construction, and struggle with adversarial obfuscation.

Parallel research has applied machine learning to malware detection at the binary and package level. Traditional malware classifiers leverage opcode sequences, API call graphs, or byte-level n-grams combined with supervised classifiers (Liang et al., 2021; Raff et al., 2018). More recent work integrates deep neural networks for end-to-end malware detection, though these methods remain vulnerable to evasion through obfuscation, packing, or adversarial perturbations. Studies such as EC2 (Chakraborty et al., 2020) highlight that ensemble techniques can improve detection rates across malware families but still face challenges in interpretability and cross-platform generalization.

While these approaches laid the foundation for learning-based vulnerability and malware detection, they are limited in extensibility and transparency. Most systems train a monolithic model to perform end-to-end classification, which makes error analysis difficult and often reduces robustness to obfuscation or distributional shifts. In contrast, our work leverages LLMs within a modular multi-agent framework, where classification is complemented by input curation and package-level aggregation. This design preserves the advantages of supervised learning while adding interpretability, reproducibility, and scalability in the context of malicious package detection.

6.3. Software supply chain security in package ecosystems

The problem of malicious packages is part of the broader challenge of securing the software supply chain. Recent incidents such as the SolarWinds compromise and large-scale injections of malicious dependencies in ecosystems like npm and PyPI have underscored the systemic risks that arise when attackers exploit trust relationships in package distribution networks (Ohm et al., 2020). Research in this space has addressed multiple layers of the supply chain, including dependency management, vulnerability propagation, and ecosystem governance.

Empirical studies have shown that dependency networks in ecosystems such as PyPI, npm, and Maven expose substantial transitive risk: a single compromised package can affect thousands of downstream projects. Taxonomies of supply chain attacks (Ohm et al., 2020) highlight that injection through public repositories is only one of several

attack vectors, alongside compromised build pipelines, stolen credentials, and malicious updates. Detection strategies, therefore, include dependency risk scoring, provenance verification, and anomaly detection across package metadata and version histories.

Despite this progress, existing approaches face challenges in scalability and robustness against adversarial behavior. Signature-based and metadata-driven defenses can be bypassed by subtle obfuscation or typosquatting, while provenance frameworks remain limited by adoption barriers and ecosystem heterogeneity. In this context, our work on LAMPS complements existing lines of defense by focusing specifically on the semantic detection of malicious logic embedded in packages, and by investigating how modular LLM-based multi-agent systems can contribute to supply chain security with interpretable and auditable decision making.

6.4. LLM-based MAS in software engineering

Recent work has begun to explore the application of multi-agent systems powered by large language models in software engineering tasks. These systems decompose complex activities into coordinated agents that collaborate via structured prompts, often outperforming monolithic pipelines. For example, CoRe (Wang et al., 2024) is a collaborative MAS for code reviewer recommendation, where agents leverage semantic representations of pull requests and reviewer expertise to improve reviewer assignment. Metagente (Nguyen et al., 2025) employs a multi-agent prompt optimization framework for summarizing GitHub README files, demonstrating improvements in text quality via ROUGE-based feedback. Other frameworks such as CAMEL (Li et al., 2023), AgentVerse (Chen et al., 2023), and CrewAI (Tauli et al., 2025) illustrate the broader potential of role-specialized agents for cooperative reasoning, planning, and task execution.

Applications of MAS in software engineering extend beyond recommendation and summarization. DocAgent (Yang et al., 2025) applies a multi-agent architecture to automated code documentation generation, while Arcadinho et al. (2024) demonstrate the use of agent collaboration for test generation and evaluation of LLM-powered tools. Ronanki et al. (2023) investigate conversational agents for requirements elicitation, showing that task decomposition improves coverage and reduces ambiguity. Beyond task-specific contributions, several surveys have recently mapped this emerging space. He et al. (2025) review LLM-based MAS across the software lifecycle and articulate open challenges. Liu et al. (2024) provided a complementary survey of multi-agent architectures and methodologies, highlighting challenges in coordination, evaluation, and communication protocols. At the systems level, Lee et al. (2025) examine strategies for reliable decision-making in MAS, comparing aggregation schemes such as majority voting, feedback loops, and arbitration. Verdecchia (2025) proposes SALLMA, a reference architecture for LLM-based multi-agent systems, emphasizing modular orchestration and contextual memory management.

Altogether, this body of research demonstrates that LLM-driven MAS are versatile and increasingly applied across diverse SE tasks, from recommendation to documentation to testing. However, the majority of these contributions focus on productivity and collaboration rather than adversarial or security-critical contexts. In contrast, our work positions multi-agent orchestration for malicious package detection, where robustness, reproducibility, and interpretability are central. By combining input curation, file-level classification, and conservative package-level aggregation, LAMPS provides an instance of MAS applied to software supply chain security. This area has received limited attention in the current literature.

7. Conclusion and future work

This paper introduced LAMPS, a multi-agent system that leverages large language models to detect malicious Python packages through coordinated, role-specialized reasoning. The architecture departs from

monolithic classification pipelines by decomposing the analysis into autonomous agents, each responsible for a semantically distinct task and orchestrated through natural language interaction. In our configuration, lightweight agents curate inputs and aggregate evidence, while a fine-tuned classifier operates at the file level. This separation improves stability under fixed seeds, keeps inputs within context limits, and yields auditable package-level decisions that can be inspected and replayed.

Experimental results show that LAMPS consistently outperforms competitive baselines, including MPHunter, RAG-based methods, and single-agent LLMs, with clear advantages in detecting obfuscated or sparsely expressed threats. Overall, the study demonstrates the feasibility of distributed LLM reasoning for malicious code detection and underscores the broader potential of multi-agent systems in secure software engineering. Beyond surpassing competitive baselines, the modular architecture enhances transparency, interpretability, and extensibility, as intermediate outputs and rationales can be audited and additional agents can be integrated without retraining the entire pipeline.

Future work will explore scaling the architecture and formalizing the design of roles and hand-offs, including ablation studies to quantify the individual contributions of input curation, file-level classification, and verdict aggregation. We plan to extend the empirical scope to larger and time-evolving corpora, as well as to other ecosystems such as npm or Maven, with appropriate adaptations of the extractor and training data. Another direction is the integration of dynamic and structural program features into the reasoning process, together with a more comprehensive efficiency model that reports latency, throughput, memory usage, and token consumption. Finally, we will investigate explanation quality by linking classifier rationales to concrete code regions and by evaluating explanation fidelity and stability across seeds, to improve interpretability and support audit and error analysis in security-critical contexts.

CRedit authorship contribution statement

Muhammad Umar Zeshan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology; **Motunrayo Ibiyo:** Software; **Claudio Di Sipio:** Writing – review & editing, Writing – original draft; **Phuong T. Nguyen:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Conceptualization; **Davide Di Ruscio:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Funding acquisition.

Data availability

The authors do not have permission to share data.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This paper has been partially supported by the MOSAICO project (Management, Orchestration and Supervision of AI-agent Communities for reliable AI in software engineering) that has received funding from the European Union under the Horizon Research and Innovation Action (Grant Agreement No. 101189664). The work has been partially supported by the EMELIOT national research project, which has been funded by the MUR under the PRIN 2020 program (Contract 2020W3A5FY). It has been also partially supported by the European Union–NextGenerationEU through the Italian Ministry of University and Research, Projects PRIN 2022 PNRR “*FRINGE: context-aware Fairness engineering in complex software systems*” grant n. P2022553SL. We acknowledge the Italian “PRIN 2022” project TRex-SE: “*Trustworthy Recommenders for Software Engineers*,” grant n. 2022LKJWHC. We thank the

anonymous reviewers for their valuable comments and suggestions that helped us improve the manuscript.

References

- Ahmad, W.U., Chakraborty, S., Ray, B., Chang, K.-W., 2021. Unified Pre-Training for Program Understanding and Generation. Technical Report. arXiv preprint.
- Alfadel, M., Costa, D.E., Shihab, E., 2023. Empirical analysis of security vulnerabilities in python packages. Proceedings of the 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) <https://doi.org/10.1109/SANER50967.2021.00048>
- Allamanis, M., Barr, E.T., Devanbu, P., Sutton, C., 2018. A survey of machine learning for big code and naturalness. ACM Comput. Surv. 51 (4). <https://doi.org/10.1145/3212695>
- Arcadinho, S., Cio, D.A., Almeida, M., 2024. Automated Test Generation to Evaluate Tool-Augmented LLMs as Conversational AI Agents. Technical Report. arXiv preprint.
- Choi, Y., Lee, J.-H., 2023. Codeprompt: task-agnostic prompt tuning for program and language generation. In: Findings of the Association for Computational Linguistics: ACL 2023. 2023. ACL Anthology paper. <https://doi.org/10.18653/v1/2023.findings-acl.325>
- Chakraborty, T., Pierazzi, F., Subrahmanian, V.S., 2020. Ec2: ensemble clustering and classification for predicting android malware families. IEEE Trans. Dependable Secure Comput. 17 (2), 262–277. <https://doi.org/10.1109/TDSC.2017.2739145>
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., 2021. Evaluating large language models trained on code. Technical Report. arXiv preprint.
- Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Qian, C., Chan, C.-M., Qin, Y., Lu, Y., Xie, R., 2023. Agentverse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors in Agents. Technical Report. arXiv preprint.
- Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. A c/c code vulnerability dataset with code changes and cve summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories (MSR), pp. 508–512. <https://doi.org/10.1145/3379597.3387501>
- Guo, W., Xu, Z., Liu, C., Huang, C., Fang, Y., Liu, Y., 2023. An empirical study of malicious code in PyPI ecosystem. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, pp. 166–177.
- He, J., Treude, C., Lo, D., 2025. LLM-based multi-agent systems for software engineering: literature review, vision and the road ahead. <https://doi.org/10.1145/3712003>
- Hq, Y., 2024. Yara forge - a rule-sharing platform for yara. <https://github.com/YARAHQ/yara-forge>.
- Huang, Y., Wang, R., Zheng, W., Zhou, Z., Wu, S., Ke, S., Chen, B., Gao, S., Peng, X., 2024. Spiderscan: practical detection of malicious NPM packages based on graph-based behavior modeling and matching. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24, Page 1146–1158. Association for Computing Machinery. <https://doi.org/10.1145/3691620.3695492>
- Ibiyo, M., Louangdy, T., Nguyen, P.T., Di Sipio, C., Di Ruscio, D., 2025. Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?. Vol. 9798400700446 of the 29th International Conference on Evaluation and Assessment in Software Engineering, EASE '25 New York, NY, USA. Association for Computing Machinery. <https://doi.org/10.1145/3756681.3757042>
- Lee, X.Y., Akatsuka, S., Vidyaratne, L., Kumar, A., Farahat, A., Gupta, C., 2025. Reliable decision-making for multi-agent LLM systems. In: Proceedings of the 2025 Workshop on Multi-Agent Systems. https://multiagents.org/2025_artifacts/reliable_decision_making_for_multi_agent_llm_systems.pdf.
- Li, G., Hammoud, H. A. A.K., Itani, H., Khizbullin, D., Ghanem, B., 2023. Camel: Communicative Agents for "Mind" Exploration of Large Language Model Society. <https://arxiv.org/abs/2303.17760>.
- Li, X., Gong, Y., Shen, Y., Qiu, X., Zhang, H., Yao, B., Qi, W., Jiang, D., Chen, W., Duan, N., 2022. Coderetriever: Unimodal and Bimodal Contrastive Learning for Code Search. CoRR, abs/2201.10866. <https://arxiv.org/abs/2201.10866>.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeepecker: a deep learning-based system for vulnerability detection. In: Proceedings of the Network and Distributed System Security Symposium (NDSS). <https://doi.org/10.14722/NDSS.2018.23158>
- Liang, G., Zhou, X., Wang, Q., Du, Y., Huang, C., 2021. Malicious packages lurking in user-friendly python package index. In: 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). IEEE, pp. 606–613.
- Liang, W., Ling, X., Wu, J., Luo, T., Wu, Y., 2023. A needle is an outlier in a haystack: hunting malicious PyPI packages with code clustering. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 307–318. <https://doi.org/10.1109/ASE56229.2023.00085>
- Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., Lou, Y., 2024. Large Language Model-Based Agents for Software Engineering: A Survey. Technical Report. arXiv preprint. <https://arxiv.org/abs/2409.02977>.
- Nettersheim, F., Arlt, S., Rademacher, M., 2024. Utilizing DNS and Virustotal for Automated Ad-Malware Detection. Vol. 14629 of Tampere, Finland. Springer. https://doi.org/10.1007/978-3-031-62362-2_31
- Nguyen, D. S.H., Truong, B.G., Nguyen, P.T., Di Rocco, J., Di Ruscio, D., 2025. Teamwork Makes the Dream Work: LLMs-Based Agents for GitHub README.MD Summarization. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3696630.3728511>
- Nguyen, P.T., Di Sipio, C., Di Rocco, J., Di Penta, M., Di Ruscio, D., 2022. Adversarial attacks to API recommender systems: time to wake up and smell the coffee? In: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, ASE '21. IEEE Press, pp. 253–265. <https://doi.org/10.1109/ASE51524.2021.9678946>
- Ohm, M., Plate, H., Sykosch, A., Meier, M., 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. Vol. 2020 of Lisbon, Portugal. Springer. DIMVA.
- Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., Nicholas, C., 2018. Malware detection by eating a whole exe. In: AAAI Workshop on Artificial Intelligence for Cyber Security (AICS). <https://arxiv.org/abs/1710.09435>.
- Robillard, M.P., Maalej, W., Walker, R.J., Zimmermann, T., 2014. Recommendation Systems in Software Engineering. Springer. <https://doi.org/10.1007/978-3-642-45135-5>
- Ronanki, K., Berger, C., Horkoff, J., 2023. Investigating ChatGPT's potential to assist in requirements elicitation processes. In: 2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). IEEE, pp. 354–361.
- Ruohonen, J., Hjerpe, K., Rindell, K., 2021a. A large-scale security-oriented static analysis of python packages in PyPI. In: 2021 18th International Conference on Privacy, Security and Trust (PST), pp. 1–10. <https://doi.org/10.1109/PST52912.2021.9647791>
- Samaana, H., Costa, D.E., Shihab, E., Abdellatif, A., 2025. A machine learning-based approach for detecting malicious PyPI packages. In: Proceedings of the 40th ACM/SI-GAPP Symposium on Applied Computing, pp. 1617–1626.
- Smith, M.Q., Ruxton, G.D., 2020. Effective use of the mcnemar test. Behav. Ecol. Sociobiol. 74, 1–9.
- Taulli, T., Deshmukh, G., Crewai, 2025. Building Generative AI Agents. Springer.
- Verdecchia, R., 2025. SALLMA: a software architecture for LLM-based multi-agent systems. In: Software Architecture Trends Conference. <https://robertoverdecchia.github.io/papers/SATrends2025.pdf>.
- Vu, D., Newman, Z., Meyers, J.S., 2022. A Benchmark Comparison of Python Malware Detection Approaches. <https://arxiv.org/abs/2209.13288>.
- Wang, B., Yue, X., Sun, H., 2023. Can ChatGPT Defend its Belief in Truth? Evaluating LLM Reasoning via Debate. Technical Report. arXiv preprint.
- Wang, L., Zhou, Y., Zhuang, H., Li, Q., Cui, D., Zhao, Y., Wang, L., 2024. Unity is strength: collaborative LLM-based agents for code reviewer recommendation. In: Proceedings of the 39th IEEE/ACM ASE, ASE '24. ACM, pp. 2235–2239. <https://doi.org/10.1145/3691620.3695291>
- Yang, D., Simoulin, A., Qian, X., Liu, X., Cao, Y., Teng, Z., Yang, G., 2025. DocAgent: A Multi-Agent System for Automated Code Documentation Generation. Technical Report. arXiv preprint. <https://doi.org/10.18653/v1/2025.acl-demo.44>
- Yang, Z., Shi, J., He, J., Lo, D., 2022. Natural attack for pre-trained models of code. In: Proceedings of the 44th International Conference on Software Engineering, ICSE '22. Association for Computing Machinery, pp. 1482–1493. <https://doi.org/10.1145/3510003.3510146>
- Zeshan, M.U., Ibiyo, M., Nguyen, P.T., Di Sipio, C., Di Ruscio, D., 2025. Replication Package: "Many Hands Make Light Work: An LLM-based Multi Agent System for Detecting Malicious PyPI Packages. <https://github.com/muzeshan/lamps-jss>.
- Zhang, J., Huang, K., Huang, Y., Chen, B., Wang, R., Wang, C., Peng, X., 2025. Killing two birds with one stone: malicious package detection in NPM and PyPI using a single model of malicious behavior sequence. ACM Trans. Software Eng. Methodol. 34 (4). <https://doi.org/10.1145/3705304>
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Adv. Neural Inf. Process. Syst. 32. <https://arxiv.org/pdf/1909.03496.pdf>.